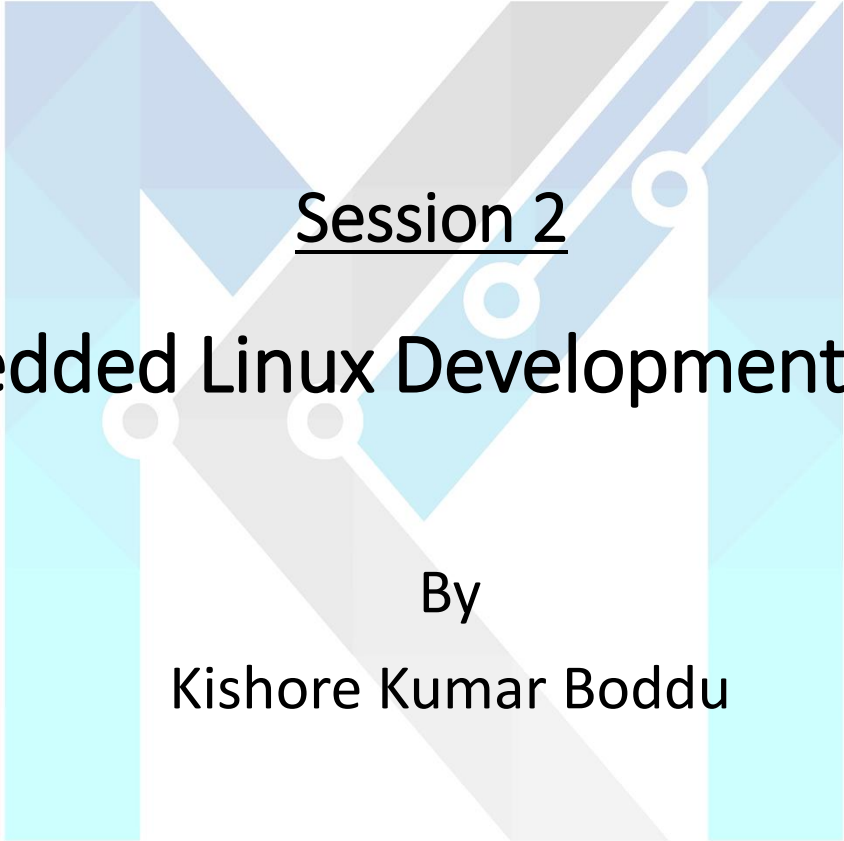


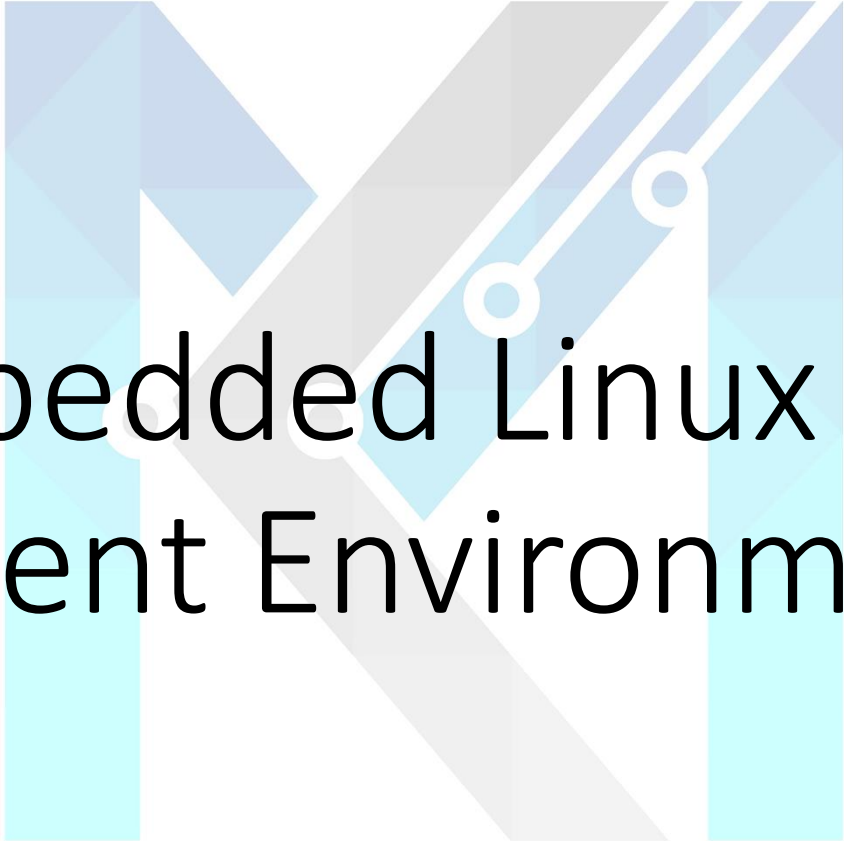


Session 2

Setup Embedded Linux Development Environment

By

Kishore Kumar Boddu



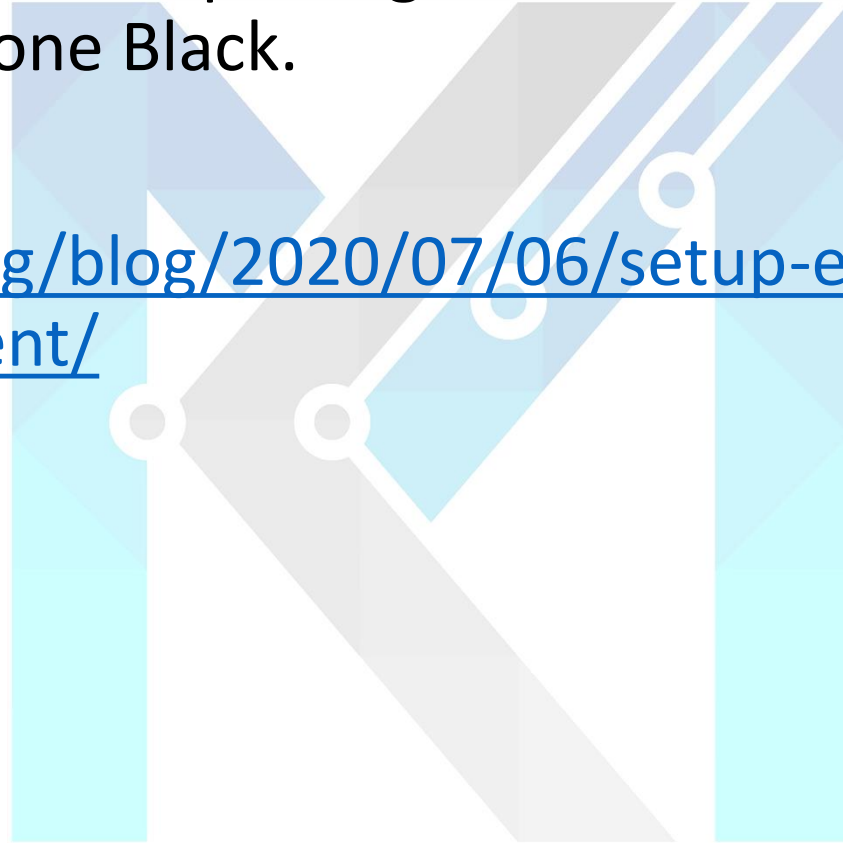
Setup Embedded Linux Development Environment

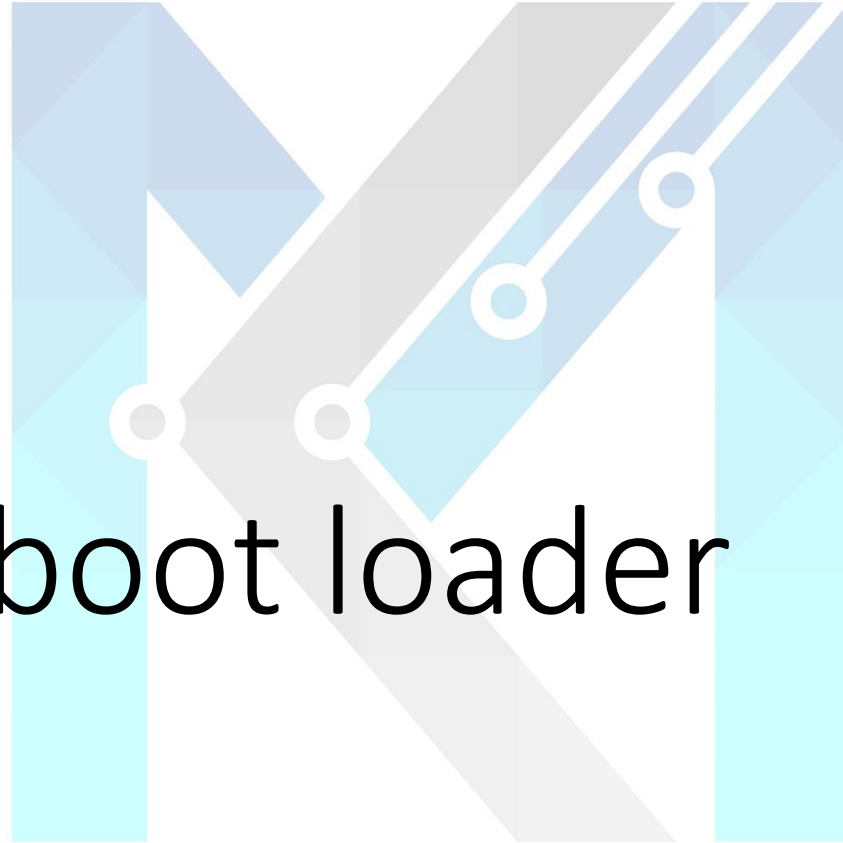
Own built images

Debian 10 own built images porting on KM-BBB

The below post explain how to porting Debian 10 own built images on Kernel Masters Beagle Bone Black.

<https://kernelmasters.org/blog/2020/07/06/setup-embedded-linux-development-environment/>





Universal boot loader

@ u-boot level

Universal Boot Loader

- U-boot Overview
- U-boot Source code layout
- U-boot Development Environment
- U-boot Initialization
- U-boot Commands
- U-boot Environment Variables
- U-boot Customization



U-Boot Overview

- U-Boot is a typical free software project
 - License: GPLv2 (same as Linux)
 - Freely available at <http://www.denx.de/wiki/U-Boot>
 - Documentation available at <http://www.denx.de/wiki/U-Boot/Documentation>
 - The latest development source code is available in a Git repository: <http://git.denx.de/?p=u-boot.git;a=summary>
 - Since the end of 2008, it follows a fixed-interval release schedule. Every two months, a new version is released. Versions are named YYYY.MM.

U-Boot Overview

- The **include/configs/** directory contains one configuration file for each supported board
 - It defines the CPU type, the peripherals and their configuration, the memory mapping, the U-Boot features that should be compiled in, etc.
 - It is a simple .h file that sets C pre-processor constants. See the README file for the documentation of these constants. This file can also be adjusted to add or remove features from U-Boot (commands, etc.).
- Assuming that your board is already supported by U-Boot, there should be one entry corresponding to your board in the boards.cfg file.

Configuring and compiling U-Boot

- **Download u-boot source code from kernel masters GITHUB server.**

```
$ cd ~/KM_GITHUB
```

```
$ git clone https://github.com/kernelmasters/beagleboneblack-uboot.git
```

- **U-boot Configuration**

```
$ cd ~/KM_GITHUB/beagleboneblack-uboot
```

```
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x_evm_config
```

- **U-boot Compilation**

```
$ cd ~/KM_GITHUB/beagleboneblack-uboot
```

```
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
```


U-boot prompt

- Connect the target to the host through a serial console.
- Power-up the board. On the serial console, you will see something like:

U-Boot 2013.04 (May 29 2013 - 10:30:21)

OMAP36XX/37XX-GP ES1.2, CPU-OPP2, L3-165MHz, Max CPU Clock 1 Ghz

IGEPv2 + LPDDR/NAND

I2C: ready

DRAM: 512 MiB

NAND: 512 MiB

MMC: OMAP SD/MMC: 0

Die ID #255000029ff800000168580212029011

Net: smc911x-0

KM=>

- The U-Boot shell offers a set of commands. We will study the most important ones, see the documentation for a complete reference or the help command.

U-boot commands

Help information

KM=> help

Version details

KM=> version

U-Boot 2013.04 (May 29 2013 - 10:30:21)

KM Boot Menu

KM=> km_bootmenu

KM Multi Boot Menu

- 1: Boot From SD Card [MMC0]
- 2: Boot From EMMC [MMC1]
- 3: Boot From Serial [UART0]
- 4: Boot From TFTP [Default CPSW]
- 5: Boot With KGDB-KDB [SDCARD,EMMC,TFTP]
- 6: Select Ethernet [CPSW,ENC]
- 7: Self diagnostic test
- 0: Stay in Boot Mode

Enter Your Choice:

Board Information

KM=> bdfinfo

arch_number = 0x00000e05

boot_params = 0x80000100

DRAM bank = 0x00000000

-> start = 0x80000000

-> size = 0x20000000

eth0name = cpsw

ethaddr = 64:cf:d9:99:5c:50

current eth = cpsw

ip_addr = 192.168.1.2

baudrate = 115200 bps

TLB addr = 0x9fff0000

relocaddr = 0x9ff56000

reloc off = 0x1f756000

irq_sp = 0x9df35ea0

sp start = 0x9df35e90

Early malloc usage: 410 / 1000

Important commands (1)

- The exact set of commands depends on the U-Boot configuration
- `help` and `help` command
- **`boot`**, runs the default boot command, stored in `bootcmd`
- **`bootm <address>`** , starts a kernel image loaded at the given address in RAM
- **`ext2load`**, loads a file from an ext2 filesystem to RAM
 - And also `ext2ls` to list files, `ext2info` for information
- **`fatload`**, loads a file from a FAT filesystem to RAM
 - And also `fatls` and `fatinfo`
- **`tftp`**, loads a file from the network to RAM
- **`ping`**, to test the network

Important commands (2)

- **loadb, loads, loady**, load a file from the serial line to RAM
- **usb**, to initialize and control the USB subsystem, mainly used for USB storage devices such as USB keys
- **mmc**, to initialize and control the MMC subsystem, used for SD and microSD cards
- **nand**, to erase, read and write contents to NAND flash
- **erase, protect, cp**, to erase, modify protection and write to NOR ash
- **md**, displays memory contents. Can be useful to check the contents loaded in memory, or to look at hardware registers.
- **mm**, modifies memory contents. Can be useful to modify directly hardware registers, for testing purposes.

Environment variables commands

- U-Boot can be configured through environment variables, which affect the behaviour of the different commands.
- Environment variables are loaded from ash to RAM at U-Boot startup, can be modified and saved back to ash for persistence
- There is a dedicated location in ash to store U-Boot environment, defined in the board configuration file.
- Commands to manipulate environment variables:
 - **printenv**, shows all variables
 - **printenv <variable-name>**, shows the value of one variable
 - **setenv <variable-name> <variable-value>**, changes the value of a variable, only in RAM
 - **saveenv**, saves the current state of the environment to flash

Important U-Boot environment variables

- **bootcmd**, contains the command that U-Boot will automatically execute at boot time after a configurable delay, if the process is not interrupted
- **bootargs**, contains the arguments passed to the Linux kernel, covered later
- **serverip**, the IP address of the server that U-Boot will contact for network related commands
- **ipaddr**, the IP address that U-Boot will use
- **netmask**, the network mask to contact the server
- **ethaddr**, the MAC address, can only be set once
- **bootdelay**, the delay in seconds before which U-Boot runs bootcmd
- **autostart**, if yes, U-Boot starts automatically an image that has been loaded into memory

Kernel & DTB images **boot from eMMC** using u-boot commands

Load kernel image from eMMC first partition boot folder:

```
KM=> load mmc 1:1 ${loadaddr} /boot/vmlinuz-4.19.94-Kernel-Masters-gc7621ff2c-dirty
```

Load dtb image from eMMC first partition dtbs folder:

```
KM=> load mmc 1:1 ${fdtaddr} /boot/dtbs/4.19.94-Kernel-Masters-gc7621ff2c-dirty/km-bbb-am335x.dtb
```

Setup Kernel command line boot arguments:

```
KM=> setenv bootargs console=${console} root=/dev/mmcblk1p1 rootfstype=${mmccrootfstype}
```

Execute kernel image with dtb image:

```
KM=> bootz ${loadaddr} - ${fdtaddr}
```


Kernel & DTB images **boot from Sdcard** using u-boot commands

Load kernel image from sdcard first partition boot folder:

```
KM=> load mmc 0:1 ${loadaddr} /boot/vmlinuz-4.19.94-Kernel-Masters-gc7621ff2c-dirty
```

Load dtb image from sdcard first partition dtbs folder:

```
KM=> load mmc 0:1 ${fdtaddr} /boot/dtbs/4.19.94-Kernel-Masters-gc7621ff2c-dirty/km-bbb-am335x.dtb
```

```
Kishore=> load mmc 0:1 ${loadaddr} /boot/vmlinuz-4.19.94-gc8fc2bfff-dirty
```

Setup Kernel command line boot arguments:

```
KM=> setenv bootargs console=${console} root=/dev/mmcblk0p1 rootfstype=${mmccrootfstype}
```

Execute kernel image with dtb image:

```
KM=> bootz ${loadaddr} - ${fdtaddr}
```


Scripts in environment variables

- Environment variables can contain small scripts, to execute several commands and test the results of commands.
 - Useful to automate booting or upgrade processes
 - Several commands can be chained using the ; operator
 - Tests can be done using
if command ; then ... ; else ... ; fi
 - Scripts are executed using `run <variable-name>`
 - You can reference other variables using `${variable-name}`
- **Example**

```
setenv mmc-boot 'mmc init 0; if fatload mmc 0 80000000 boot.ini; then  
source; else if fatload mmc 0 80000000 ulmage; then run mmcbootargs;  
bootm; fi; fi'
```

Script example

```
'if ping 192.8.1.1; then;  
    echo "ping success";  
else ;  
    echo "ping fail";  
fi
```

=> setenv test 'if ping 192.8.1.1; then; echo "ping success"; else ; echo "ping fail"; fi'

=> run test

Kernel & DTB images boot from eMMC using script

```
#define KM_UENV_EMMC "setenv bootcmd "\
    "echo board_name=[${board_name}] ...;"\
    "echo \"***** Booting From EMMC .... *****\";"\
    "echo board_name=[${board_name}] ...;"\
    "setenv fdtfile km-bbb-am335x.dtb;" \
    "setenv console ttyO0,115200n8;"\
    "load mmc 1:1 ${loadaddr} /boot/uEnv.txt;"\
    "env import -t ${loadaddr} ${filesize};" \
    "echo uname_r=[${uname_r}] ...;"\
    "echo board_no=[${board_no}] ...;"\
    "load mmc 1:1 ${loadaddr} /boot/vmlinuz-${uname_r};"\
    "load mmc 1:1 ${fdtaddr} /boot/dtbs/${uname_r}/km-bbb-am335x.dtb;"\
    "setenv emmc emmc;"\
    "setenv bootargs console=${console} root=/dev/mmcblk1p1 rootfstype=${mmicrootfstype};"\
    "echo ****bootz:Start Kernel****;"\
    "bootz ${loadaddr} ${rdaddr}:${rdsiz} ${fdtaddr};"\
    "echo ***END***;"
```

Transferring files to the target

- U-Boot is mostly used to load and boot a kernel image, but it also allows to change the kernel image and the root filesystem stored in flash.
- Files must be exchanged between the target and the development workstation. This is possible:
 - Through the network if the target has an Ethernet connection, and U-Boot contains a driver for the Ethernet chip. This is the fastest and most efficient solution.
 - Through a USB key, if U-Boot supports the USB controller of your platform
 - Through a SD or microSD card, if U-Boot supports the MMC controller of your platform.
 - Through the serial port.

What is Board Support Package (BSP)?

- A Board support package (BSP) is essential code for a given computer hardware device that will make that device work with the computers Operating System (OS).
- The BSP Contains a small program called a boot loader or boot manager the places the OS and device drivers into memory.
- The Content of the BSP depend on the particular hardware and OS.
- Specific tasks that the BSP performs include the following, in order:
 - Initialize the processor.
 - Initialize the bus.
 - Initialize the Interrupt Controller.
 - Initialize the clock.
 - Initialize the RAM settings.
 - Run the bootloader.
- In addition, a BSP can contain directives, compilation parameters, and hardware parameters for configuring the OS.

U-boot Source code Layout

/arch	Architecture specific files
/arc	Files generic to ARC architecture
/arm	Files generic to ARM architecture
/m68k	Files generic to m68k architecture
/microblaze	Files generic to microblaze architecture
/mips	Files generic to MIPS architecture
/nds32	Files generic to NDS32 architecture
/nios2	Files generic to Altera NIOS2 architecture
/openrisc	Files generic to OpenRISC architecture
/powerpc	Files generic to PowerPC architecture
/riscv	Files generic to RISC-V architecture
/sandbox	Files generic to HW-independent "sandbox"
/sh	Files generic to SH architecture
/x86	Files generic to x86 architecture
/api	Machine/arch independent API for external apps
/board	Board dependent files

/cmd	U-Boot commands functions
/common	Misc architecture independent functions
/configs	Board default configuration files
/disk	Code for disk drive partition handling
/doc	Documentation (don't expect too much)
/drivers	Commonly used device drivers
/dts	Contains Makefile for building internal U-Boot fdt.
/examples	Example code for standalone applications, etc.
/fs	Filesystem code (cramfs, ext2, jffs2, etc.)
/include	Header Files
/lib	Library routines generic to all architectures
/Licenses	Various license files
/net	Networking code
/post	Power On Self Test
/scripts	Various build scripts and Makefiles
/test	Various unit test files
/tools	Tools to build S-Record or U-Boot images, etc.

X-loader (SPL) Initialization Flow

Processor Specific Initialization:

`arch/arm/cpu/armv7/start.S:`

reset: -> cpu_init_cp15 -> cpu_init_crit -> lowlevel_init

`arch/arm/cpu/armv7/lowlevel_init.S:` (setup pll,mux,memory)

lowlevel_init ->

For boards with SPL this should be empty since SPL can do all of this init in the SPL **board_init_f()** function which is called immediately after this.

bl s_init

SoC Specific Initialization:

`arch/arm/mach-omap2/am33xx/board.c:` [PRCM]

board_init_f: -> hw_data_init() -> early_system_init() -> sdram_init();

board_early_init_f() ->

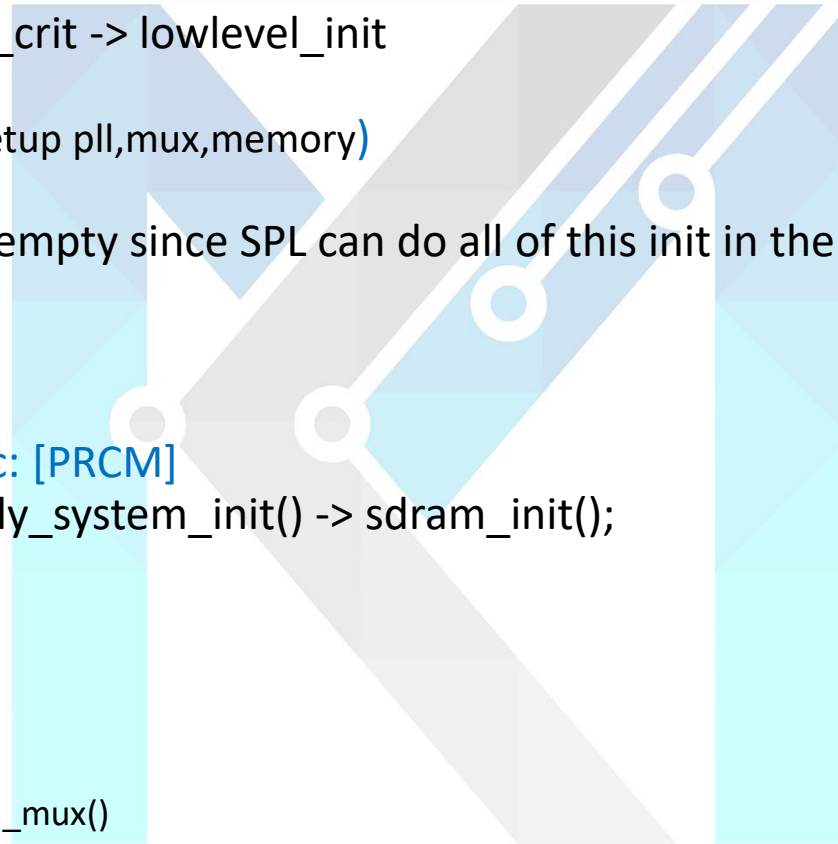
prcm_init() -> set_mux_conf_regs()

Board Specific Initialization:

`board/ti/am335xx/board.c:`

- set_mux_conf_regs() -> enable_board_pin_mux()

`board/ti/am335x/mux.c:`



U-Boot Initialization Flow

`common/main.c:`

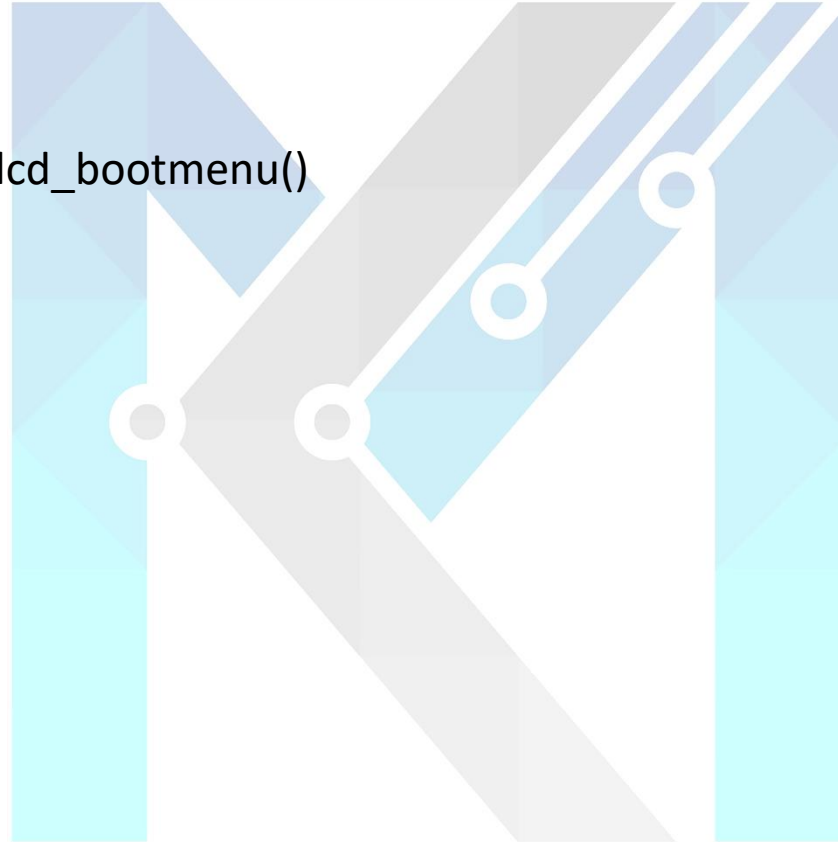
`Main_loop() -> autoboot_command()`

`common/autoboot.c:`

`Autoboot_command() -> multiboot() -> lcd_bootmenu()`

`board/ti/am335x/km_lcd.c:`

`lcd_bootmenu() ->`



U-boot Source code Initialization

